AGENT SKILLS HACKATHON · BEST USE OF GEMMA 4

ScamShield AI

Know a job scam before you pay.

An offline job-scam detector for Pakistan's job seekers: quoted red flags, local AI reasoning, screenshot reading and on-device learning — with no message ever leaving the laptop.

0
messages sent to a server — works with the Wi-Fi off

12
rule families scoring the text, links, mailboxes and numbers

44
automated tests covering the engine, guardrails, OCR and learning

PROJECT REPORT	Architecture, functions, libraries, problems and solutions, limitations
REPOSITORY	github.com/Aabyyss/ScamShield-AI
STACK	Python 3.14 · Streamlit · Ollama (Gemma 4 E2B) · RapidOCR/ONNX · SQLite
STATUS	Working end to end · 44 automated tests passing
DATE	30 October 2026

Student hackathon project. ScamShield gives guidance, not legal advice, and never declares a company criminal.

Contents

1. Executive summary
2. The problem
   - 2.1 Human problem
   - 2.2 Technical problem
   - 2.3 Non-goals
3. What the user experiences
4. Architecture
   - 4.1 Request lifecycle
5. Module reference
   - 5.1 Public functions that matter
6. Feature catalogue
   - 6.1 Verdict and scoring
   - 6.2 Red-flag taxonomy (12 families)
   - 6.3 Entity analysis (offline reputation substitute)
   - 6.4 Screenshot input
   - 6.5 Learning
   - 6.6 Report export
   - 6.7 Interface
7. Technology stack
   - 7.1 Why Gemma 4, specifically
   - 7.2 Deliberately rejected
8. How the guardrails work
9. Self-learning design
   - 9.1 Rule precision
   - 9.2 Taught phrases and trusted domains
   - 9.3 Message memory
   - 9.4 Safety properties (each covered by a test)
10. Problems encountered and how they were solved
11. Testing
    - 11.4 What the "dataset" actually is — and is not
12. Performance (measured on the development laptop)
13. Packaging and one-click launch
14. Drawbacks and limitations
    - 14.1 Functional
    - 14.2 Security and privacy
    - 14.3 Engineering
15. Roadmap
16. What this project demonstrates
17. References

WHERE TO READ IF YOU HAVE FIVE MINUTES

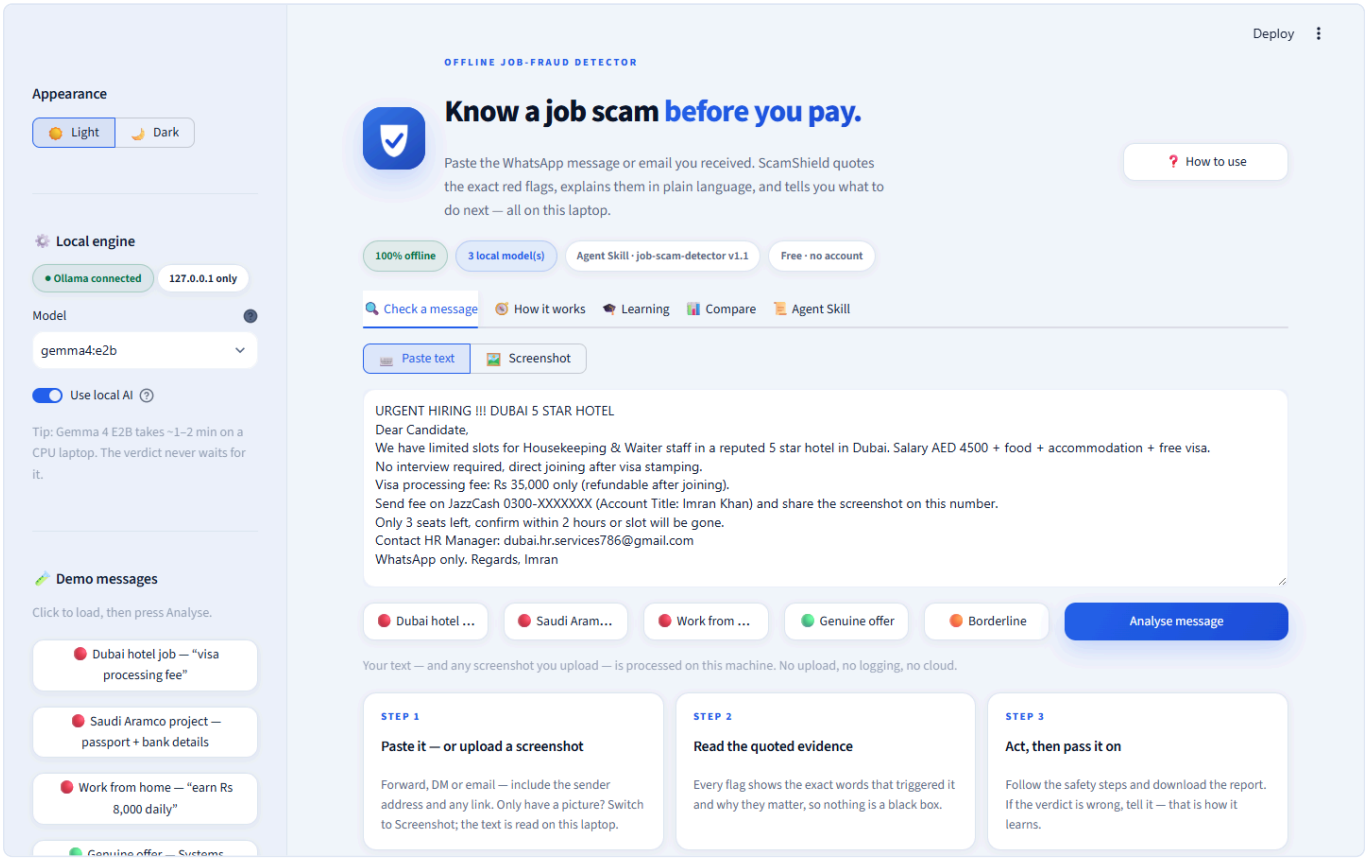
The pitch (§1 – §3)
What the problem costs a student, and what the user actually sees on screen.

How it is built (§4 – §7)
Request lifecycle, every module, the rule taxonomy, and the stack with the reasons behind each choice.

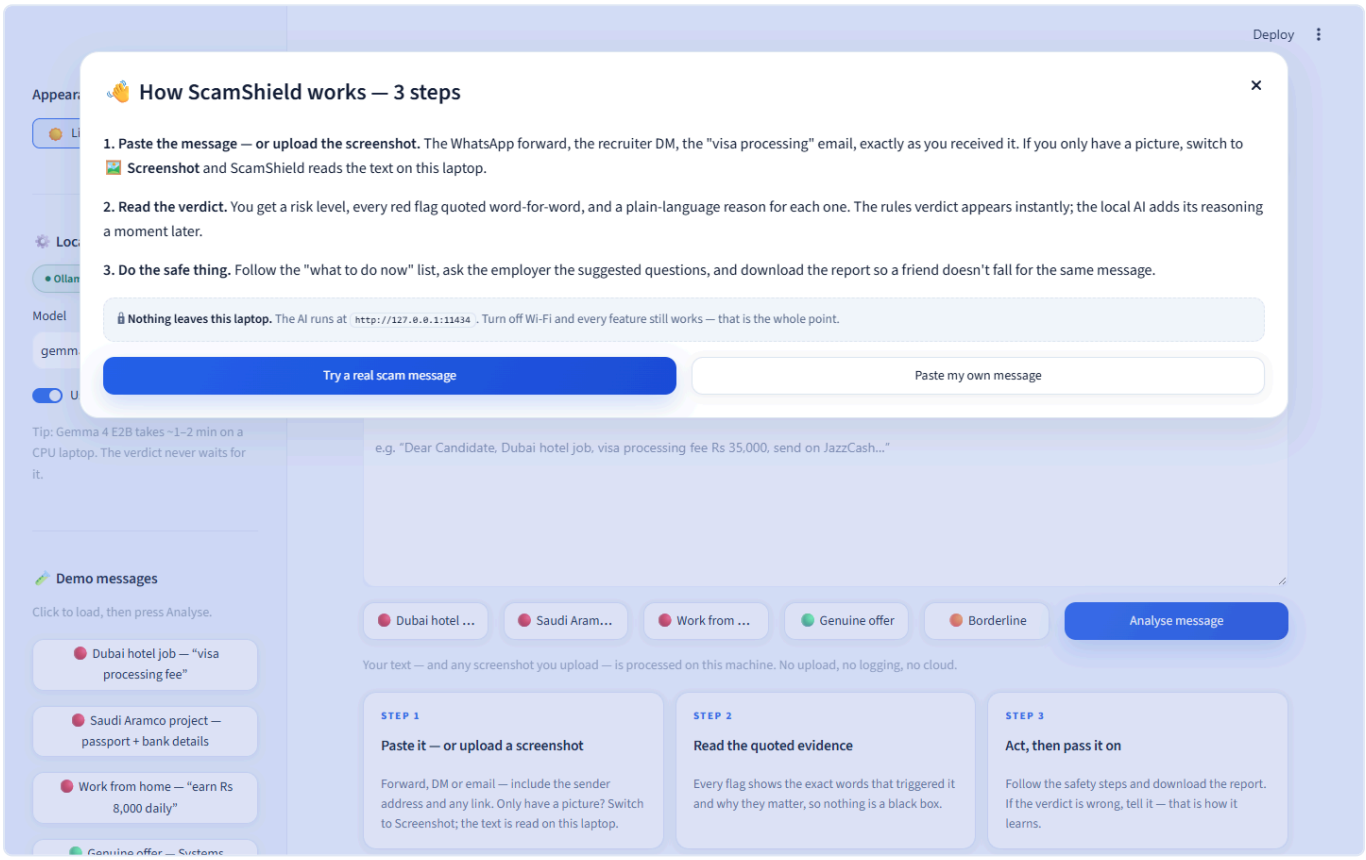
Why the verdict holds up (§8 – §11)
Guardrails against a small local model, the learning maths, the 15 problems hit while building, and the tests.

Honest limits (§12 – §17)
What was measured, what was not, what the project cannot do, and what we would build next.

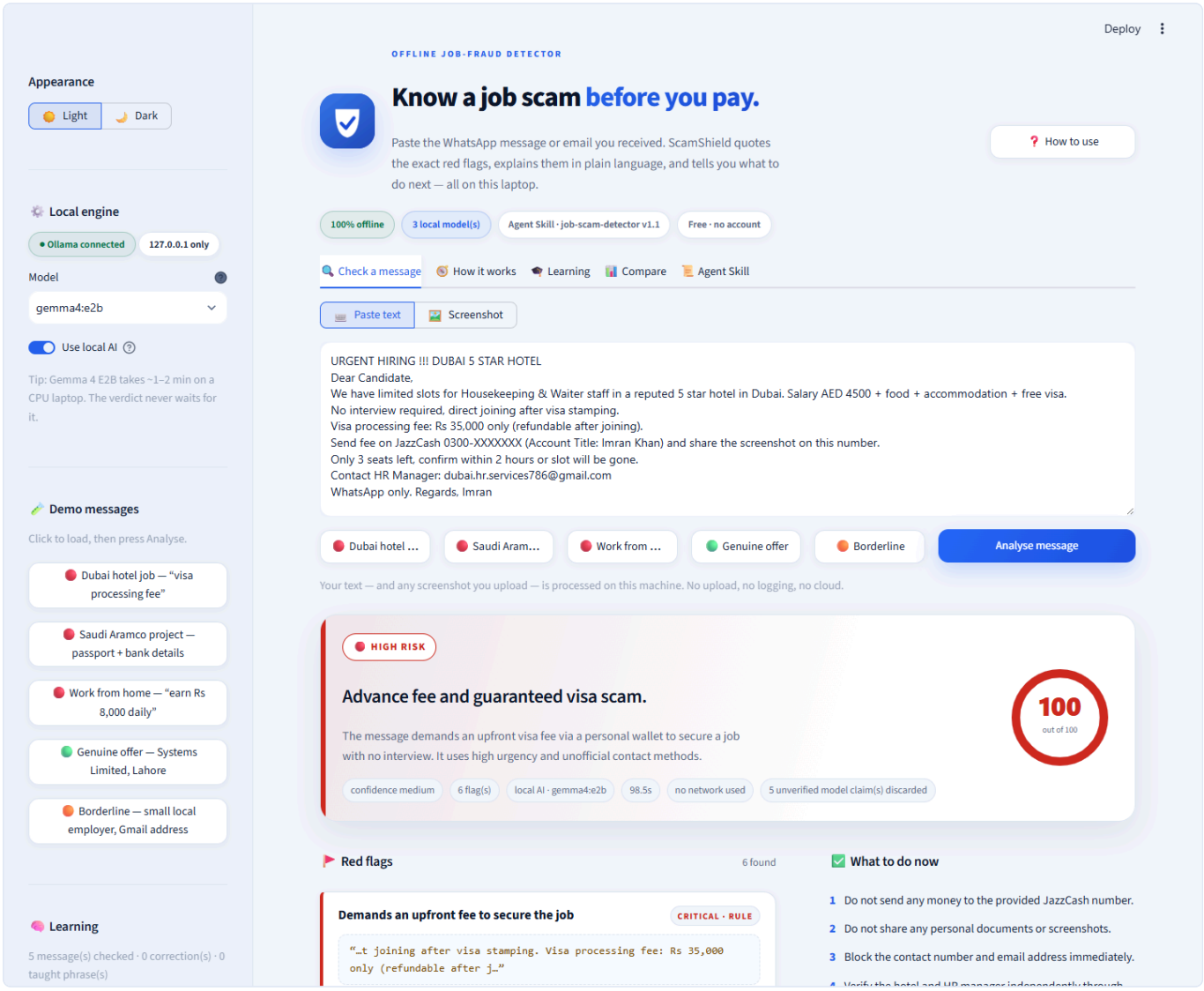
Interface



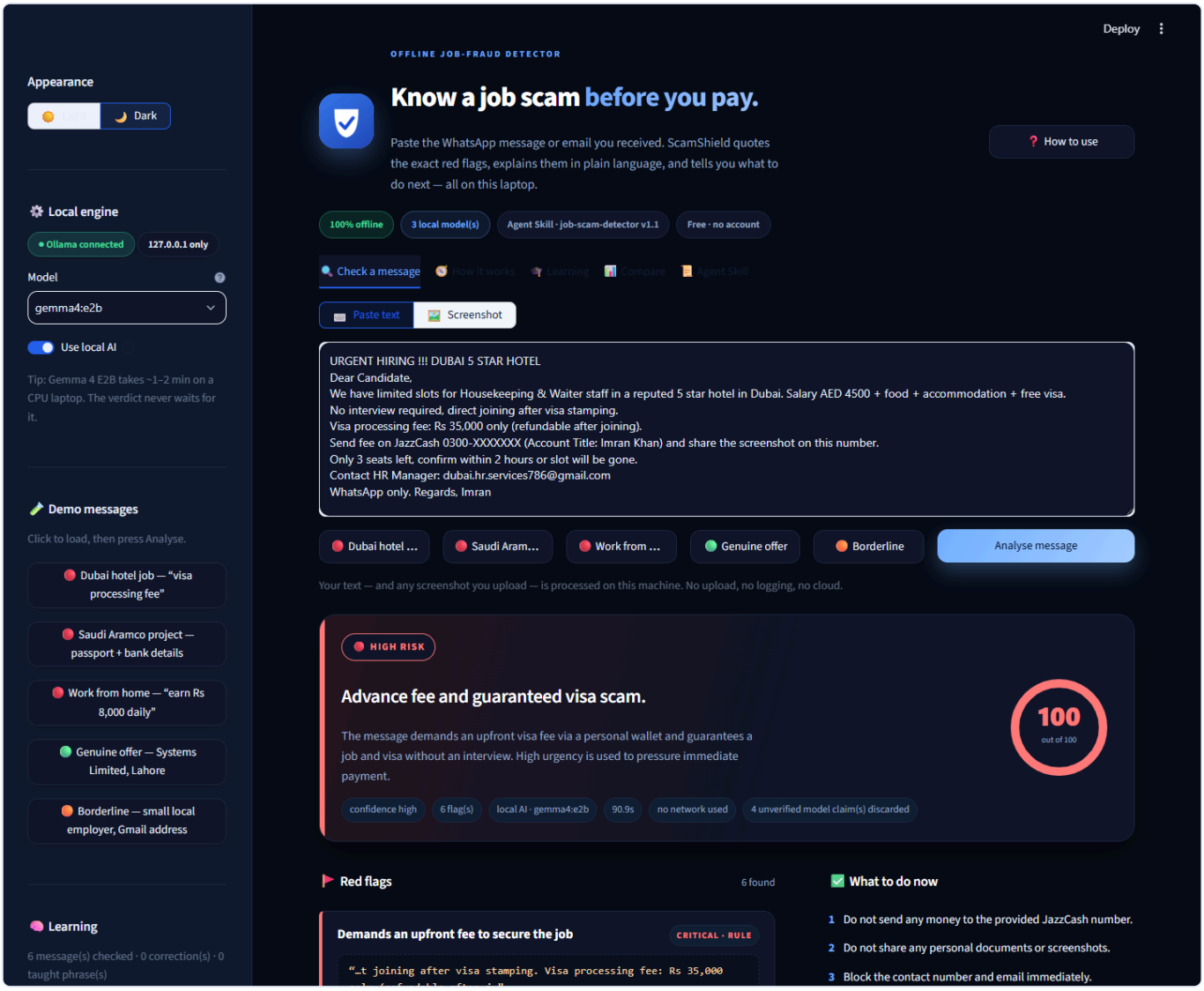
Home screen with the light theme, demo messages and quick examples



Three-step onboarding dialog shown on first launch



A scam verdict: score ring, quoted red flags and next steps



The same verdict in the dark theme

An offline job-scam detector for Pakistan's job seekers

Project	ScamShield AI — The Job Scam Detector
Repository	github.com/Aabyyss/ScamShield-AI
Built for	Agent Skills hackathon — *Best use of Gemma 4* category
Report date	30 October 2026
Stack	Python 3.14 · Streamlit · Ollama (Gemma 4 E2B) · RapidOCR/ONNX · SQLite
Tests	44 automated tests, all passing
Status	Working end to end; verified on a CPU-only Windows laptop

---

1. Executive summary

Every day, students in Pakistan lose their savings to fake overseas job offers on WhatsApp. The message looks official: a Dubai hotel, a Saudi project, a work-from-home task. It asks for a "visa processing fee" through JazzCash, promises a guaranteed visa, and asks for a CNIC and passport scan. By the time the victim understands, the money is gone and their documents are being reused.

The obvious fix — paste the message into a cloud chatbot — fails twice. Victims won't upload a message that contains their CNIC and bank details, and a general chatbot answers with prose, not evidence.

ScamShield is a desktop app that reads the message **entirely on the user's machine**. It returns a risk level, quotes the exact words that triggered every red flag, judges each link, email address and phone number in the text, and hands the user a downloadable report they can forward to warn the next person. It runs with Wi-Fi switched off, costs nothing per check, and gets measurably better as the user corrects it.

The core design decision is that **the scoring is deterministic and the language model only explains**. A small edge model running on a CPU is not trustworthy enough to decide whether someone should send money, but it is very good at turning evidence into plain language. Separating those two jobs is what makes the verdict defensible.

---

2. The problem

2.1 Human problem

Who	Students and fresh graduates in Pakistan, often first-time job seekers, using WhatsApp as their main channel
What happens	A recruiter offers an overseas job, then asks for a visa/processing/medical fee. A second variant harvests passport, CNIC and bank details. A third is the "daily earning task" scam that extracts deposits.
Why it works	The sums are small enough to feel plausible (Rs 1,000–35,000), the promise is large, the deadline is short, and there is no easy way to check a foreign employer
Why existing tools fail them	Cloud scam checkers require uploading sensitive text; Pakistani-specific signals (JazzCash, EasyPaisa, CNIC, "visa stamping") are poorly covered; general chatbots produce confident prose without evidence
Cost of a false negative	Entire savings, plus identity theft from shared documents
Cost of a false positive	A real job opportunity is lost, and the user stops trusting the tool

2.2 Technical problem

Build a scam classifier that is:

1. **Private by construction** — no message content leaves the device, provable by turning off Wi-Fi.
2. **Explainable** — every flag must quote the message and state why it matters.

3. **Robust to a weak model** — a small local model will hallucinate; the architecture must assume it.
4. **Fast enough to use** — the verdict cannot wait 90 seconds for the model.
5. **Improvable without retraining** — the user's corrections must change future behaviour, safely.
6. **Simple to run** — a non-technical user should double-click one icon.

2.3 Non-goals

- Not a legal or financial authority; it never declares a company criminal.
- No cloud calls, no accounts, no telemetry — even when that means giving up live URL reputation.
- Not a general spam filter for all message types; the taxonomy is job-fraud specific.

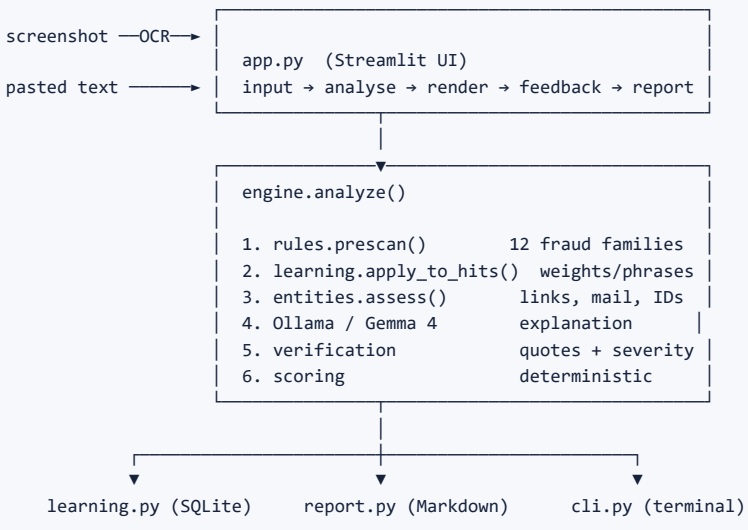
---

3. What the user experiences

1. **Open** — double-click the desktop shortcut. The launcher starts the local server and opens the browser; the first run also installs dependencies and wakes Ollama.
2. **Onboarding** — a three-step welcome dialog explains what to paste, what they will see, and why it is private. It can be reopened from ❓ **How to use**.
3. **Input** — paste text, or switch to 🖼️ **Screenshot** and upload a picture. OCR extracts the text into an editable box so mistakes can be fixed before analysis.
4. **Verdict** — the rules engine answers in milliseconds: risk band, score out of 100, and each red flag with its quote. A spinner notes that the local model is still reading.
5. **Reasoning** — sixty to ninety seconds later the summary, prioritised actions and "ask them this" questions are filled in, and the result is stored.
6. **Act** — follow the advice, download the Markdown report, or correct the verdict.
7. **Teach** — a correction updates the rules' weights; a phrase can be taught in one line.

---

4. Architecture



4.1 Request lifecycle

1. The UI calls `engine.analyze(message, use_ai=False)` first — a rules-only pass.
2. That pass runs `prescan` (regexes), then `learning.apply_to_hits` (feedback weights, taught phrases, trusted-domain dampening), then `rule_score`.
3. The verdict is rendered immediately. The extracted entities (`entities.assess`) render with it.

4. If AI is enabled and Ollama is reachable, `engine.analyze(..., use_ai=True)` runs: - `call_model` streams from `http://127.0.0.1:11434`, stopping when the JSON object balances. - The response is parsed, and every model flag is verified before use. - Rules keep their flags; the model may add only *new* taxonomy families. - Rules certify severity; model findings add at most 15 points.
5. The finished `Analysis` is stored (fingerprint only), rendered, and offered for feedback.

---

5. Module reference

FILE	RESPONSIBILITY
`app.py`	Streamlit UI: tabs, onboarding dialog, theme switch, OCR upload, two-phase analysis, feedback controls, report download
`scamshield/ui.py`	Light and dark palettes, all CSS, HTML builders (verdict ring, flag cards, entity rows, tables, metrics)
`scamshield/rules.py`	12 deterministic fraud families; each hit carries quote, severity, weight; scoring with floors
`scamshield/entities.py`	Offline extraction and risk judgement of URLs, emails, domains, phones, CNIC numbers
`scamshield/ocr.py`	Screenshot reading via RapidOCR, Tesseract fallback, image validation and upscaling
`scamshield/engine.py`	Ollama client (streaming), JSON contract, guardrails, merge logic, scoring, fallback
`scamshield/learning.py`	SQLite feedback store, Beta-smoothed rule weights, taught phrases, trusted domains, simhash memory
`scamshield/report.py`	Markdown safety report with quotes, entities, actions and disclaimer
`scamshield/samples.py`	Five-message demo corpus with expected verdicts
`scamshield/cli.py`	Terminal interface for developing the skill without the UI
`launcher.py`	One-click launcher: dependency check, Ollama wake-up, free port, detached server, browser open
`ScamShield.bat`	Windows double-click entry point with error pause
`setup_desktop.bat` + `tools/create_shortcut.ps1`	Creates the desktop shortcut with the project icon
`launch.sh`	macOS/Linux entry point
`tools/make_logo.py`	Generates the logo and multi-size `.ico` from code
`skills/job-scam-detector/SKILL.md`	The Agent Skill: investigator playbook, taxonomy, JSON contract, compact runtime brief
`tests/`	44 regression tests across rules, entities, learning, report and OCR

5.1 Public functions that matter

```
# scamshield/engine.py
analyze(message, model="", use_ai=True, use_learning=True) -> Analysis
call_model(message, hits, model, timeout=300) -> dict
load_skill() -> str                      # full SKILL.md
runtime_brief() -> str                   # 320-word projection sent to the model

# scamshield/rules.py
prescan(text) -> list[Hit]               # every rule hit with quoted evidence
rule_score(hits) -> int                  # 0-100, critical floor at 82

# scamshield/entities.py
assess(text, trusted=None) -> list[Entity]
domains_in(text) -> list[str]

# scamshield/learning.py
record_verdict(text, result) -> int
record_feedback(verdict_id, predicted, actual, rule_keys, note="") -> list[dict]
apply_to_hits(hits, message) -> (list[Hit], list[str])
rule_multipliers() -> dict[str, float]
similar_cases(text) -> list[SimilarCase]
add_phrase(phrase) / add_trusted_domain(domain) / reset_learning()

# scamshield/ocr.py
extract(image_bytes) -> OCRResult       # text, engine, confidence, warnings

# scamshield/report.py
build_report(result, message) -> str
```

6. Feature catalogue

6.1 Verdict and scoring

- **Three bands** — HIGH ≥ 65, MEDIUM ≥ 30, LOW below 30.
- **One decisive signal wins** — any critical family floors the score at 82, so a message cannot look "mostly fine" while demanding a visa fee.
- **Score is arithmetic**, not model sentiment: rule weights plus at most 15 points for verified model findings.
- **Confidence** is reported separately and never fabricated from the score.

6.2 Red-flag taxonomy (12 families)

FAMILY	SEVERITY	WEIGHT
Upfront fee to secure the job	critical	45
Payment to a private wallet (JazzCash, USDT, account title)	critical	45
Guaranteed visa / no interview / direct joining	critical	40
Identity harvesting (passport, CNIC, bank statement, OTP)	critical	40
Daily-pay task scam	critical	35
Look-alike, throwaway or shortened link	high	28
Pay far above market	high	24
Free mailbox or WhatsApp-only contact	medium	20
Skipped hiring process	high	22
Manufactured urgency	medium	16
Employer never named	medium	18
Bulk copy-paste blast	low	12

Every hit stores up to three quoted windows around the match, so the UI can show evidence rather than a label.

6.3 Entity analysis (offline reputation substitute)

SIGNAL	VERDICT	REASON GIVEN TO THE USER
Gmail/Yahoo/Outlook sender	high	Anyone can create one; real HR writes from the company domain
Brand look-alike (`careers-systemsltd.com`)	high	Uses Systems Limited's name but is not `systemsltd.com`
URL shortener	high	Real destination hidden until click
Punycode domain	high	Can display as a different name
Disposable TLD (`.xyz` , `.top` , `.tk` ...)	medium	Cheap, disposable, common in fraud runs
Hyphenated recruiter domain	medium	Typical throwaway recruiter pattern
Partly masked phone (`0300-XXXXXXX`)	medium	Unverifiable contact
CNIC number	info	Shown masked; "never send this to an unverified employer"
Ordinary phone number	info	Cannot check offline; call the published number instead

6.4 Screenshot input

RapidOCR (ONNX) runs inside the process. Images are EXIF-corrected, upscaled to 1600 px so word spacing survives (below that, `Send fee on JazzCash` collapsed into one token and broke the rules), capped at 12 MB, and the extracted text lands in an editable box. Confidence is shown; below 0.6 the app warns that words may be wrong.

6.5 Learning

Covered in detail in §8. In the UI: a **Learning** tab with per-rule precision, taught phrases, trusted domains, recent checks and two reset buttons.

6.6 Report export

A Markdown file containing the verdict, score, every flag with its quote, actions, questions, entity findings, similar past messages, reporting contacts, disclaimer, and the original message inside a collapsible block (so the user keeps their own record).

6.7 Interface

- Light theme by default (projector-friendly), dark theme as a full second palette.
- Onboarding dialog, empty-state guide, quick example chips.
- Comparison tab with competitor capabilities and sources.
- Everything offline: no fonts, images or scripts fetched from the internet.

---

7. Technology stack

TOOL / LIBRARY	VERSION	USED FOR	WHY THIS ONE
Python	3.14	Everything	Available on the laptop; fast enough; huge library support
Streamlit	1.66.1	UI	Fastest path to a working, styled app; built-in dialog, segmented control, file uploader, download button
Ollama	local service	Model runtime	One-command local models, streaming API, keeps the model warm, no Python ML stack needed
Google Gemma 4 E2B	3.5 GB (q4)	Explanation layer	The target model for the *Best use of Gemma 4* category. E2B is the edge variant, so it fits an 8 GB CPU laptop; E4B / 12B / 26B-A4B run unchanged on a bigger machine
requests	2.31+	Ollama HTTP client	Streaming support with a simple API
SQLite (`sqlite3`)	stdlib	Learning store	Zero setup, transactional, single file, perfect for a desktop app

TOOL / LIBRARY	VERSION	USED FOR	WHY THIS ONE
Pillow	13.0	Image handling, logo generation	Reliable decoding, resizing, EXIF handling, and drawing the logo/icon without design tools
RapidOCR (ONNX)	1.2.3	Screenshot OCR	Ships its own models, pure pip install, CPU inference, no system binary or network
ONNX Runtime	1.31.1	OCR inference	Cross-platform, fast on CPU
OpenCV	5.0.0.93	RapidOCR dependency	Image preprocessing inside the OCR pipeline
unittest	stdlib	Tests	No extra dependency; runs with one command
PowerShell + WScript.Shell	Windows	Desktop shortcut	Native `.lnk` creation with a custom icon
Git / GitHub	—	Version control	Collaboration and submission

7.1 Why Gemma 4, specifically

PROPERTY	WHY IT MATTERS HERE
Edge-first E2B / E4B variants	The judge's laptop is the deployment target, not a GPU box. E2B runs offline on 8 GB of RAM, which is what makes the "turn off the Wi-Fi" demo possible at all
Structured JSON output	The engine asks for one JSON object and validates every field; Gemma 4 follows the schema closely enough that the guardrails rarely have to throw an answer away
Multimodal input (text + image)	The screenshot path already exists via RapidOCR. Gemma 4 can read the image directly, which is the obvious next step for handwriting and Urdu script — we do **not** claim it today, because the current release routes images through OCR
Runs through Ollama	No Python ML stack, no PyTorch download, one command to install, and the model stays warm between checks

Model choice is a one-line change: `SCAMSHIELD_MODEL=gemma4:e4b` or the sidebar dropdown. Nothing in the scoring or the guardrails depends on which Gemma 4 size is used.

7.2 Deliberately rejected

OPTION	WHY NOT
Cloud LLM API (OpenAI, Gemini)	Breaks the central privacy claim and adds cost per check
Tesseract	Needs a system binary; not installed and would fail on a judge's laptop
EasyOCR / PaddleOCR	Pulls PyTorch (hundreds of MB, slow install, fragile on Python 3.14)
scikit-learn classifier	Rules are more explainable and need no training data; a learned model would be a black box
Vector database / embeddings	No corpus; rules + model cover the need without infrastructure
Electron / Tauri front end	Weeks of work for a 24-hour hackathon; Streamlit delivers the same UX
Live domain reputation API	Requires network; would undermine the offline demo

---

8. How the guardrails work

The model is treated as an unreliable narrator. Its output passes four gates:

1. **Taxonomy or nothing.** Each flag must carry a `code` from the twelve families. Free-form accusations are discarded.
2. **Quote or nothing.** `evidence` must appear in the message — exact match after normalisation, or ≥75% token overlap with at least three content tokens. A fabricated quote kills the flag.
3. **No invented payment.** A payment claim is dropped when its own quote denies payment. This check exists because the model *did* flag a genuine offer letter ("no payment or fee of any kind is required") as a payment scam during development.
4. **Rules certify severity.** `critical` / `high` model claims are downgraded to `medium` unless a rule matched the same family. Model findings add at most 15 points.

Additional structural safeguards:

- Rules own the flags for any family they matched; the model may only add **new** families. This removed incoherent duplicate flags produced by the 1B model the project started on.
- A LOW verdict cannot carry an alarmist headline or a scam-worded summary.
- Discarded claims are counted and shown in the result footer.

---

9. Self-learning design

9.1 Rule precision

For each rule that fired, the store keeps `scams` and `legit` counts from user corrections. Precision is smoothed with a Beta prior:

```
precision = (2 + scams) / (4 + scams + legit)
multiplier = clamp(0.55 + 0.9 × precision, 0.55, 1.45)   # only when scams + legit ≥ 3
```

At the prior mean (0.5) the multiplier is exactly 1.0, so no correction changes anything until three labels accumulate. A rule confirmed on three scams reaches ×1.19; a rule wrongly firing on three legitimate offers drops to ×0.81.

9.2 Taught phrases and trusted domains

- A phrase must be at least two words, so "visa" cannot be taught. A match contributes one clearly-labelled 22-point flag, separate from built-in rules.
- A trusted domain dampens the free-mailbox and link heuristics by 75% and drops their severity, but **never** removes a critical payment or document signal.

9.3 Message memory

Each scan stores a 64-bit simhash (3-token shingles, BLAKE2b). Similarity is Hamming distance; ≤ 8 bits is treated as near-duplicate. The message text itself is never stored, which is why the app can claim privacy while still saying "you checked 3 messages like this; two were scams".

9.4 Safety properties (each covered by a test)

- One correction cannot move a weight.
- Re-labelling a verdict replaces its old evidence instead of stacking it.
- Feedback cannot whitewash a critical signal: after three "legitimate" labels on a Dubai-fee message, the verdict stays HIGH with a score ≥ 82.
- Everything is visible and resettable; no hidden state.

---

10. Problems encountered and how they were solved

These are real failures from the build, not hypotheticals.

#	SYMPTOM	ROOT CAUSE	SOLUTION	EVIDENCE
1	CLI crashed printing results on Windows	Console is cp1252; output contains `→` and `“”`	Reconfigure stdout to UTF-8 at CLI start	CLI prints cleanly
2	Genuine offer flagged as identity harvesting	Rule reacted to "bring your CNIC copy" (normal at a Pakistani interview)	Require a *send/share/upload* verb near the document, not mere mention	`test_bringing_cnic_to_interview_is_not_harvesting`

#	SYMPTOM	ROOT CAUSE	SOLUTION	EVIDENCE
3	Genuine Systems Limited offer returned **HIGH**	The Gemma 3 1B model we started on invented a payment demand for a letter that says "no fee"	Entire verification layer: taxonomy codes, quote verification, no-fee polarity check, severity certification, 15-point cap	`legit_offer` now LOW; `test_invented_payment_flag_is_discarded`
4	Dubai analysis returned malformed JSON after 118 s	Model hit `num_predict` cap mid-object (`done_reason: length`)	Stream and stop at the balanced closing brace; salvage complete fields if truncated	JSON parses; latency down
5	Every call took ~4 minutes	Prompt was the full 2,524-token playbook	Added a 320-word `runtime:` section inside SKILL.md, parsed at runtime	Prompt ~320 words; 76–126 s on Gemma 4 E2B
6	Model produced incoherent flags (wrong code/title pairs, duplicates)	A 1B model cannot follow a large schema reliably	Rules own flags; model may only add new families; its prose is kept	Dubai flags now coherent
7	Model could lower a critical verdict	Model verdict was trusted	Rules assign score; model adds ≤15 points; critical floor at 82	`test_model_flags_never_lower_a_rule_verdict`
8	Correcting the same verdict inflated rule statistics	`ON CONFLICT` updated the feedback row but added stats again	Reverse the previous label's contribution, then apply the new one	`test_repeated_feedback_replaces_instead_of_stacking`
9	Test suite failed deleting temp DBs on Windows	SQLite connections were never closed	`_session()` context manager: commit, close in `finally`	44/44 pass, temp dirs clean up
10	Gemma 4 returned "Ollama returned an empty response" after 180 s	Gemma 4 is a reasoning model: it streams its chain of thought into a separate `message.thinking` field, and on CPU that reasoning used the whole token budget before any JSON was emitted	Send `think: false` in the Ollama payload; if `content` is still empty, salvage the JSON from `thinking`	`engine=ollama` on all five samples, 76–126 s
11	Risk badges rendered dark instead of red	Global `.stMarkdown span` colour rule outranked component classes	Narrowed the global selector to structural text only	Computed badge colour now `#cf2a20` / `#ff7a7a`
12	OCR collapsed word spaces (`SendfeeonJazzCash`)	RapidOCR loses spacing on small renders; rules match on `\b` boundaries	Upscale to 1600 px before recognition	900 px → collapsed; 1600 px → correct sentence
13	New dev server said "port 8501 is not available"	A stale Streamlit process still held the port	Launcher records PID/port in `data/launcher.json` , reuses a healthy server, and reports conflicts	`launcher.py --restart` verified
14	Random `AttributeError` in dataclasses during editing	Streamlit hot-reload re-imported modules mid-save	`runOnSave = false` in config; refresh instead	Stable sessions

#	SYMPTOM	ROOT CAUSE	SOLUTION	EVIDENCE
15	Download button key collisions	Same widget key across reruns with different results	Key includes the verdict id	No duplicate-key errors

11. Testing

44 tests, all passing, run with:

```
python -m unittest discover -s tests -v
```

AREA	WHAT IS ASSERTED
Demo corpus	Each of the five samples lands in its expected band; legit offer has no critical flags
False positives	"CNIC at interview" and "no fee at any stage" do not trigger harvesting/payment rules
Scoring	Band thresholds, 100-point cap, critical floor
Model guardrails	Invented payment discarded, uncorroborated HIGH downgraded, model cannot lower a verdict, quote verification with and without ellipsis
Entities	Free mailbox, shortener, lookalike brand, official domain, masked phone, masked CNIC, domain extraction, clean text
Learning	One label changes nothing, three labels move weight, confirmed scams raise weight, replacement semantics, critical floor survives feedback, phrase validation, phrase flag, trusted-domain dampening, reset, near-duplicate detection, no message text in the DB
Report	Verdict, quote and disclaimer present
OCR	Printed screenshot read correctly, blank image warns, invalid bytes raise a readable error, oversized file rejected, OCR text flows into the detector

Not covered by tests: UI interactions (verified manually in the browser), model output quality (no golden LLM snapshots — it is non-deterministic), real WhatsApp screenshots (synthetic renders are used), load or concurrency testing.

11.4 What the "dataset" actually is — and is not

This project does **not** have a dataset in the statistical sense, and the report should not be read as claiming measured accuracy.

WHAT EXISTS	SIZE	ORIGIN	CAN SUPPORT AN ACCURACY CLAIM?
Demo corpus (`scamshield/samples.py`)	5 messages (90, 94, 74, 120, 75 words)	Written by the team to exercise the 12 rule families	**No**
Unit tests	44 tests	Hand-written inputs, including fragments of the demo corpus	**No**
OCR fixtures	2 rendered PNG cards	Generated with Pillow, not captured from WhatsApp	**No**
Real user messages	0	—	—

The five messages cover three scam variants, one genuine offer letter and one borderline case. They are realistic in style and Pakistan-specific detail, but they were **written alongside the rules**. That makes the corpus checks regression tests — they prove the rules still fire as designed — rather than held-out evaluation. A model tuned this way can look perfect on its own corpus and still miss a real message with different wording.

What *is* genuinely verified: the deterministic layers behave as specified on known inputs; the guardrails block specific model failures (invented payment, unverified quote, severity inflation, verdict lowering); OCR reads printed Latin text; no message text reaches the database. Those are correctness properties, not accuracy measurements.

Known coverage bias: English-first wording, Pakistan-specific payment rails (JazzCash, EasyPaisa), overseas-employment framing, and printed text. Roman-Urdu, Urdu script, purely local job scams, and novel fee structures are underrepresented or absent.

What a defensible evaluation would need:

1. 200+ consented, anonymised real messages from WhatsApp, email and DMs, with personal data removed before storage.
2. Two independent annotators labelling each message as scam / suspicious / legitimate against a written rubric, with disagreements adjudicated.
3. A held-out split never used while editing rules or prompts.
4. Reporting precision, recall and F1 per band, plus the **false-positive rate on legitimate offer letters**, which is the number that decides whether users trust the tool.
5. Separate results by channel and language, and a real-screenshot OCR word-error rate.

The local feedback store already captures the labels a user assigns to real messages. With consent and anonymisation, those corrections are the natural seed for that dataset.

---

12. Performance (measured on the development laptop)

CPU-only, no GPU, 8 threads, Ollama 0.40, Python 3.14:

PATH	TIME	NOTES
Rules + entities only	~0.007 s	Rendered immediately; CLI measured 7 ms
`gemma4:e2b` full verdict	76–126 s (median 94 s)	Default model, measured over the five demo messages; Gemma 4 at q4, thinking disabled
`llama3.2` (3B) full verdict	~4 min	Better prose, unusable on stage
Screenshot OCR	3.3 s module / 6.3 s in UI	70-word card at 91% mean confidence
Model warm-up	~16 s	Done in a background thread at startup
Launcher reuse of a running server	0.3 s	Reads state, checks `/healthz`, opens browser

Optimisations that mattered: instant rules-first paint, the compact runtime brief, stopping generation at the closing brace, keeping the model loaded for 30 minutes, and 1600 px OCR upscaling (accuracy worth ~3 s).

---

13. Packaging and one-click launch

FILE	PURPOSE
`launcher.py`	Resolves the project root, installs requirements on first run, wakes Ollama, picks a free port, starts Streamlit detached, waits for `/healthz`, opens the browser, saves state
`ScamShield.bat`	Double-click entry with a readable failure pause
`setup_desktop.bat` + `tools/create_shortcut.ps1`	Creates `ScamShield AI.lnk` on the Desktop, targeting `pythonw.exe launcher.py` with `assets/icon.ico`
`launch.sh`	macOS/Linux entry point
`assets/logo.png`, `assets/logo-192.png`, `assets/icon.ico`	App logo, page icon and multi-size Windows icon
`tools/make_logo.py`	Reproducible logo generation (gradient tile, shield, checkmark)

Launcher flags: `--port`, `--no-browser`, `--restart`, `--stop`. A collaborator clones the repo, runs `setup_desktop.bat` once, and gets the same one-click experience; `ScamShield.bat` works without any setup.

---

14. Drawbacks and limitations

14.1 Functional

LIMITATION	IMPACT	MITIGATION
AI layer takes 65–95 s on CPU	The explanation lags the verdict	Rules verdict is instant; the UI says the model is still reading
Small local models are weak	Gemma 4 E2B is far stronger than the 1B model we began with, but it is still not a frontier model and can write noisy prose	Guardrails; flags and score come from rules
OCR reads printed Latin text well, handwriting and Urdu poorly	A photographed Urdu message may fail	Editable transcription box; confidence shown; Urdu OCR on roadmap
No live reputation data	Cannot know a domain was reported yesterday	Structural checks; the UI states reputation was not checked
Brand-lookalike list is 14 companies	New brands are missed	Generic hyphen/free-mail/risky-TLD rules still fire
English-first rules	Roman-Urdu messages may pass	Taught phrases partially compensate; Roman-Urdu rules on roadmap
Learning is per-install and single-user	No population-level improvement	Explicitly a personalisation feature, not shared intelligence

14.2 Security and privacy

LIMITATION	IMPACT	MITIGATION
No authentication, single-user desktop model	Not safe for shared or hosted deployment	Bind to `127.0.0.1`; documented as a desktop app
Learning DB is unencrypted	Local level/fingerprint history readable by the OS user	No message text stored; CNIC masked in output
Trusted-domain allowlist can be abused	A scammer using a trusted domain gets dampened signals	Dampening only; critical families keep their floor; documented as a preference, not proof
Report includes the pasted message	Sharing the report shares the text	User-initiated only; message is inside a collapsible block
Feedback could be poisoned	A malicious user could dampen a rule	Single-user threat model; bounded weights; reset; critical floor
Shortcut is unsigned	Windows SmartScreen may warn on a downloaded `.bat`	Expected for hackathon code; source is visible

14.3 Engineering

LIMITATION	IMPACT	MITIGATION
Simhash similarity is coarse	Possible false "seen this before" matches	8/64-bit threshold; last 500 scans only; presented as context, not evidence
Streamlit is a single process	Concurrent sessions serialise model calls	Desktop use; OCR inference is locked
Launcher is Windows-first	macOS/Linux get `launch.sh`, no desktop icon	Documented
Shortcut stores the interpreter path	Moving Python or deleting `.venv` breaks it	Re-run `setup_desktop.bat`
Tests do not cover the UI or model quality	Regressions possible in those areas	Manual browser checks; deterministic layers fully tested
Evaluation corpus is 5 synthetic messages	No accuracy, precision or recall claim is possible	Stated openly in §11.4; real-message evaluation is the next step

LIMITATION	IMPACT	MITIGATION
No PDF export, no messaging integration	Cannot act inside WhatsApp	Downloadable Markdown report
Python 3.10+ required	Slightly narrows compatibility	Stated in README

15. Roadmap

1. **Urdu and handwritten OCR** — the most common real input for the target user.
2. **Roman-Urdu rule patterns** (`paisay bhejain`, `visa guarantee`, `ghar baithe kamai`).
3. **Local reputation cache** from messages the user has already judged, labelled as cached and possibly stale.
4. **One-page PDF** generated from the Markdown report.
5. **Employer verification helper** — an offline checklist and official-domain list extension.

16. What this project demonstrates

- A privacy-preserving AI product can be built **without** a cloud model, and the constraint becomes the demo: switching off Wi-Fi proves the claim.
- With a small model, the winning architecture is **rules decide, model explains**, with explicit verification gates rather than trust.
- "Self-learning" can be honest at this scale: bounded statistical updates, visible evidence, reversible actions — not a claim of retraining a neural network.
- Explainability is a UX feature: quoted triggers, exposed weights and discarded claims make the verdict arguable, which is what users need before sending money.

17. References

- Bitdefender Scamio — product and FAQ pages: https://www.bitdefender.com/en-us/consumer/scamio
- Norton Genie FAQ: https://support.norton.com/sp/de/de/home/current/solutions/v20230717145233467
- ScamAdviser algorithm explainer: https://www.scamadviser.com/articles/scamadviser-algorithm-explainer
- Truecaller Scam Checker: https://www.truecaller.com/scam-checker
- Google Messages scam protection: https://support.google.com/messages/answer/9327903
- FTC job-scam guidance: https://consumer.ftc.gov/articles/job-scams
- Beta-binomial smoothing reference: https://bookdown.org/kevin_davisross/stat415-handouts/beta-binomial.html
- RapidOCR: https://github.com/RapidAI/RapidOCR
- Ollama API: https://github.com/ollama/ollama/blob/main/docs/api.md
- Streamlit documentation: https://docs.streamlit.io

This report describes the state of the repository at commit `main` on 30 October 2026. It is a student hackathon project, not a security product; treat its verdicts as a second opinion.